ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Stream

04/21/2025

Before we get started…

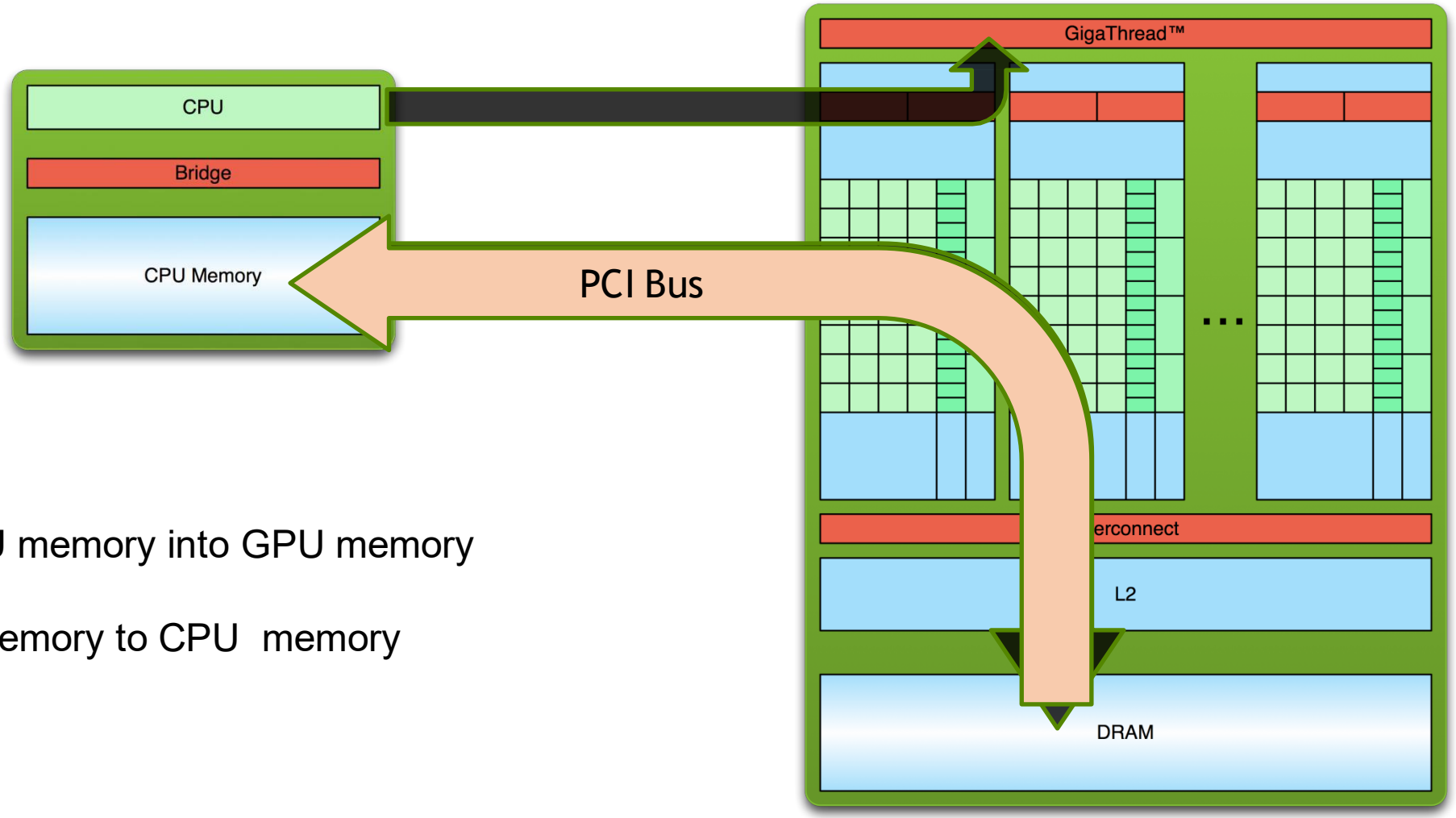
- Quick overview, things discussed last time
  - GPU-parallel scan
- Purpose of today's lecture
  - GPU task concurrency using streams
- Miscellaneous
  - Programming assignment #7 is released – due **Friday 5/2 at 23:50 PM**
  - Take-home final exams will take place during the week of 4/25 – 5/4
  - In-class final project presentation take place on 4/25, 4/28, 5/2 – schedule available here
    - You will need to upload your slides by noon on your scheduled presentation day
    - Link will be available on Canvas
  - Please complete course evaluation: https://it.wisc.edu/services/heliocampusac/
    - Bonus question in final exam: how many questions did you answer?

CUDA Streams

What is a CUDA Stream?

- CUDA stream allows you to add a GPU operation into to a *queue* that will be scheduled to run in a first-come first-serve order by the CUDA runtime
  - A GPU operation can be a kernel, memory copy, or other CUDA-supported API call

The “GPU computing” Process



1. Copy input data from CPU memory into GPU memory
2. Launch a GPU Kernel
3. Copy results from GPU memory to CPU memory
4. Repeat Many Times

[Justin Luitjens - NVIDIA]→

Why do We Need CUDA Streams?

- Modern GPU typically has at least two engines to run scheduled GPU operations
  - An execution engine to run kernel operations
  - A copy engine to run data transfer operations simultaneously, with two additional sub-engines
    - A H2D copy sub-engine
    - A D2H copy sub-engine
- Stream allow us to utilization these engines and program “task-level parallelism”
- Goal of the “streams” here: Learn how to simultaneously use both GPU engines
  - Unlike kernel, which is more on algorithm-level innovation, stream is more on scheduling-level innovation
  - Properly using streams allows more efficient overlap of GPU operations

Asynchronous Concurrent Execution: most CUDA calls are asynchronous

- To facilitate concurrent execution on host and device, most CUDA calls are asynchronous
  - Control is returned to the host thread before the device has completed the requested task

- Examples of asynchronous calls include the following common operations:
  - Kernel launches
  - device ↔ device memory copies
  - host ↔ device memory copies of a memory block of 64 KB or less
  - Memory copies performed by functions that are suffixed with Async (e.g., `cudaMemcpyAsync`)

- CAVEAT: When an application is run via a (`cuda-gdb`, `nvprof`), all launches are synchronous

cudaMemcpyAsync function call

- In general, host ↔ device data transfers using `cudaMemcpy()` are blocking
  - Control is returned to the host thread only after the data transfer is complete

- There is a non-blocking variant suffixed with Async – `cudaMemcpyAsync()` :

```
cudaMemcpyAsync(a_d, a_h, size, cudaMemcpyHostToDevice, stream);
myKernel<<<grid,block>>>(a_d);
cpuFunction();  // overlap with both data copy and kernel
```

- NOTE: Asynchronous transfer version requires pinned host memory (allocated with `cudaHostAlloc()`), and it contains an additional argument (a stream ID)
  - The `cudaHostAlloc()` replaces `malloc()` typical call on the host side

Overlapping Host ↔ Device Data Transfer with Device Execution

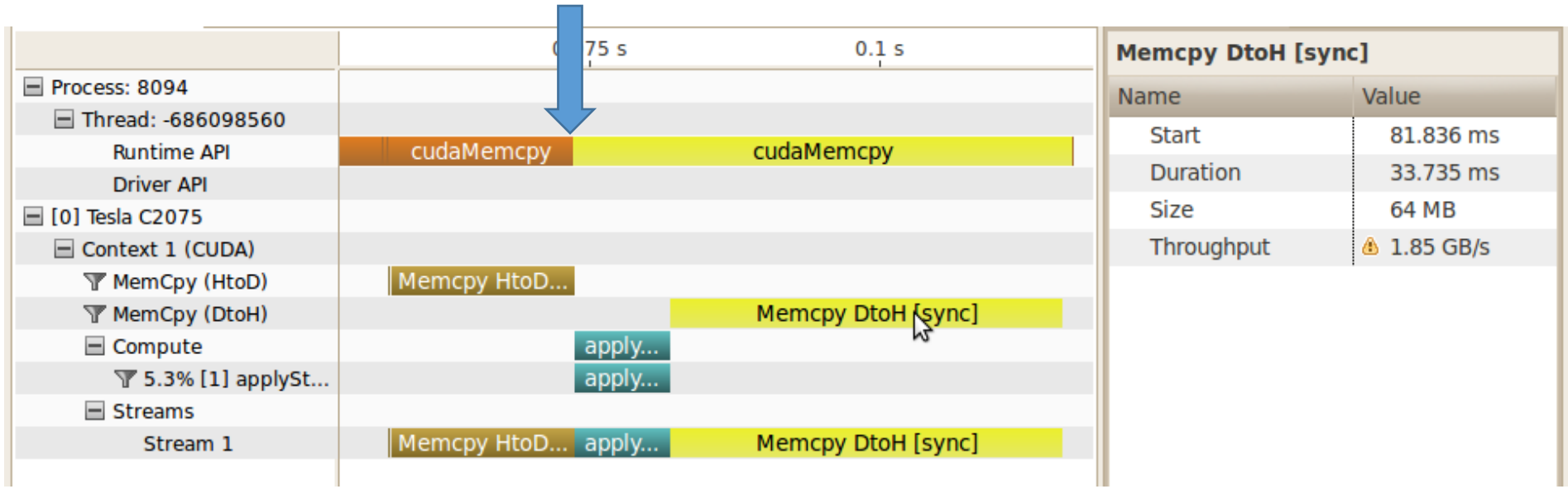
- The device execution order is first-come first-serve using a FIFO:
  - **Within a stream**, one enqueued device function is not serviced until all the previous device function calls completed
- Rule above prevents overlapping kernel execution with data transfer within same stream.
- Getting around this limitation: use two different streams
  - In one stream, you copy data from the host to device, or from the device to the host, or device to device
  - In another stream, a kernel is executed on the GPU
    - Results of the previous kernel can be copied back to CPU using another stream (task overlap)
  - Modern GPU normally has two execution engines and schedulers for kernel execution and data transfers
    - Using different streams for kernel launches and data transfers enable efficient task overlap

The Stencil Operation Kernel with Synchronous Memcpy

```
int main() {
  int wsize = 2 * RADIUS + 1;
  //allocate resources
  float *weights = new float[wsize];
  float *in = new float[N];
  float *out= new float[N];
  initializeWeights(weights, RADIUS);
  initializeArray(in, N);
  float *d_weights;  cudaMalloc(&d_weights, wsize * sizeof(float));
  float *d_in;       cudaMalloc(&d_in,      N * sizeof(float));
  float *d_out;      cudaMalloc(&d_out,     N * sizeof(float));

  cudaMemcpy(d_weights, weights, wsize, cudaMemcpyHostToDevice);
  cudaMemcpy(d_in, in, size, cudaMemcpyHostToDevice);

  applyStencil1D<<<N / 512, 512>>>(RADIUS, N-RADIUS, d_weights, d_in, d_out);

  cudaMemcpy(out, d_out, size, cudaMemcpyDeviceToHost);

  //free resources
  delete[] weights; delete[] in; delete[] out;
  cudaFree(d_weights);  cudaFree(d_in);  cudaFree(d_out);
}
```

Two Contiguous `cudaMemcpy` Operations on CPU

There is nothing after the kernel launch, but the CPU need to wait before it starts the copy for the kernel to get done, although the kernel launch is asynchronous.



Overview of CUDA Streams

- A programmer can explicitly program GPU task concurrency through *streams*

- "concurrency" refers to one of two things
  1. The "data movement" and the "execution" engines of the GPU working at the same time
  2. Several different kernels (like `kerA`, `kerB`, etc.) being executed at the same time on the GPU with potential overlaps with other data transfer operations

- One host thread can establish & manage multiple CUDA streams

- What are the typical operations in a stream?
  - Invoking a data transfer
  - Invoking a kernel execution
  - Handling events

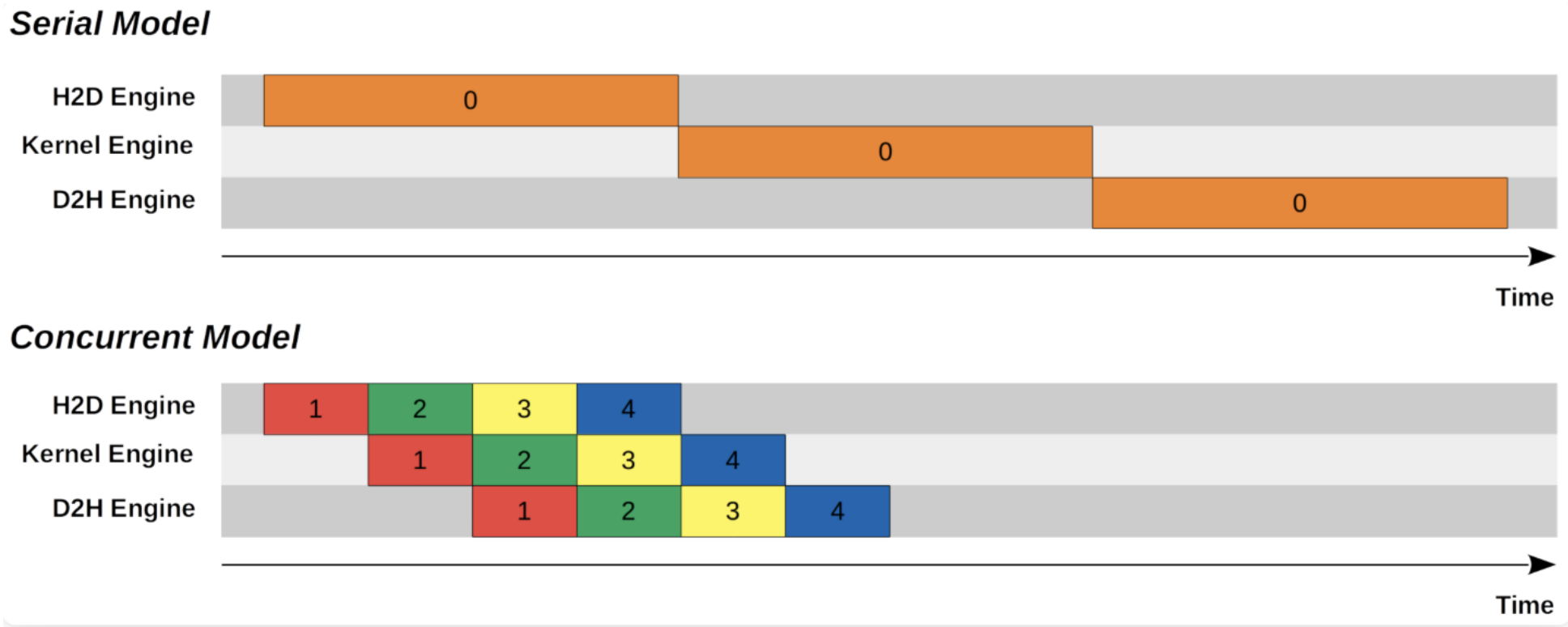
Overview of CUDA Streams (cont'd)

- A stream is a sequence of host-issued CUDA calls that execute on the GPU in FIFO order
  - CUDA stream: a queue of GPU operations; i.e., of device work/operations
  - The execution order in a stream is defined by the order in which the GPU operations are enqueued into the stream organized in first-in first-out (FIFO) order
  - An operation in a stream does not run until prior operations fully complete
    - Intra-stream, you have a "sequential consistency" model

- The CUDA runtime executes commands from **different** streams as it sees fit
  - CUDA operations in different streams may run concurrently
    - Inter-stream, you have a "task-parallel" model
  - CUDA operations from different streams are typically interleaved (in a non-deterministic fashion)
  - No inter-stream relative synchronization unless the user explicitly takes care of synchronization process
  - Streams can be synchronized at any point

Wait… So far, we have been running kernels without streams...

- As soon as you invoke a CUDA function you create a default stream (stream 0)
  - CUDA runtime always creates a default stream for you
  - If you don't explicitly state a stream in the execution configuration of a kernel it is assumed it's launched as part of "Stream 0" (default stream)



Example, Part 1 – CUDA Streams: Creation

- A stream is defined by creating a stream object using `cudaStreamCreate`
  - It is subsequently used by specifying it as the stream parameter to a sequence of kernel launches and host-to/from-device memory copies

- The following code sample creates two streams and allocates an array "`hostPtr`" of `float` in page-locked (pinned) memory
  - `hostPtr` will be used in asynchronous host ↔ device memory transfers
  - Assume we would like to transfer `size` bytes of float from each stream

```
cudaStream_t stream[2];
for (int i = 0; i < 2; ++i)
        cudaStreamCreate(&stream[i]);
float* hostPtr;
cudaMallocHost(&hostPtr, 2 * size);
```

Example, Part 2 – CUDA Streams: Making Use of Them

- In the code below, each of the two streams is defined as a sequence of
  - One memory copy from host to device,
  - One kernel launch, and
  - One memory copy from device to host

```
for (int i = 0; i < 2; ++i) {
  cudaMemcpyAsync(
    inputDevPtr + i * size, hostPtr + i * size, size, cudaMemcpyHostToDevice, stream[i]
  );

  MyKernel<<<100, 512, 0, stream[i]>>>(outputDevPtr+i*size, inputDevPtr + i*size, size);

  cudaMemcpyAsync(
    hostPtr + i * size, outputDevPtr + i * size,size, cudaMemcpyDeviceToHost, stream[i]
  );
}
```

Example, Part 3 – CUDA Streams: Clean Up Phase

- Streams are properly destroyed by calling `cudaStreamDestroy()`

```
for (int i = 0; i < 2; ++i)
    cudaStreamDestroy(stream[i]);
```

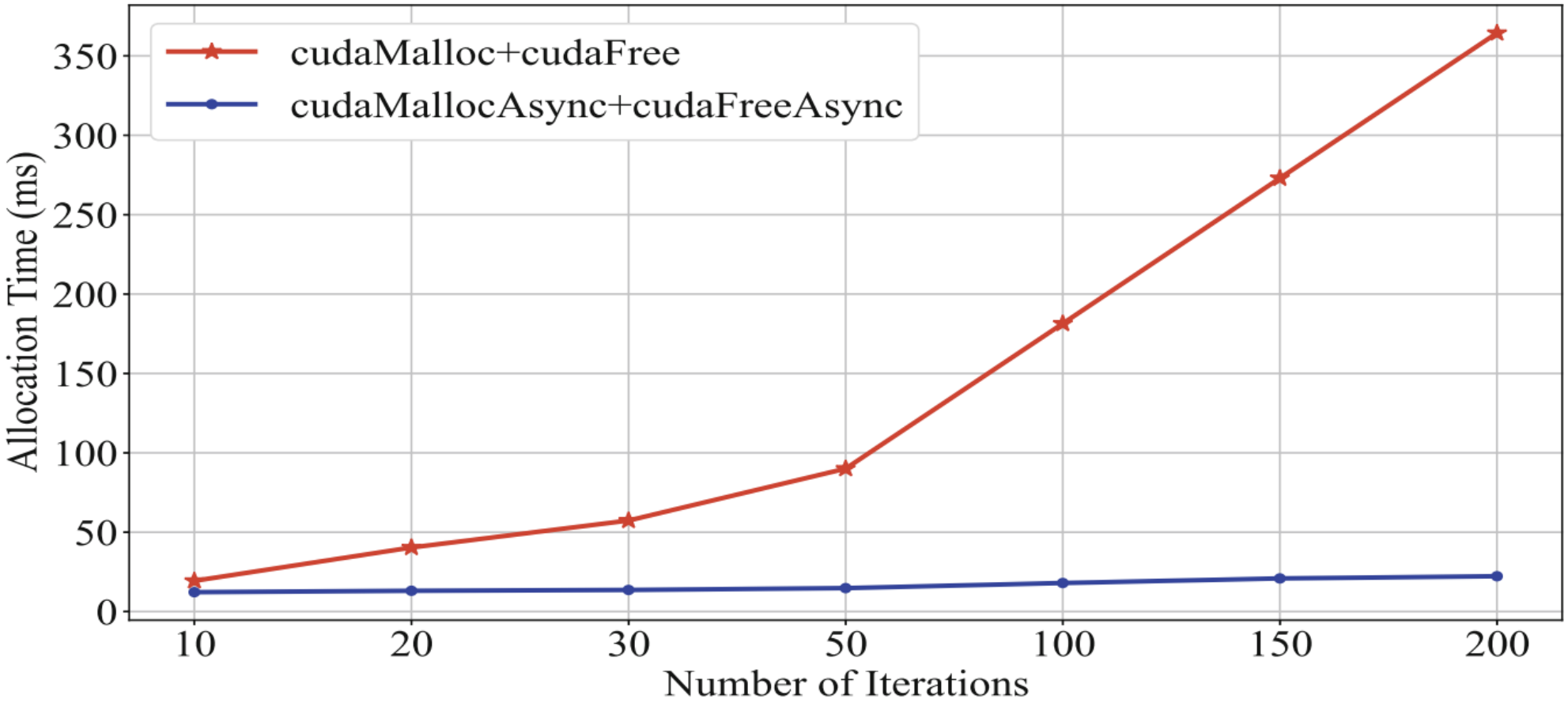
- `cudaStreamDestroy()` waits for all preceding commands in the given stream to complete before destroying the stream and returning control to the host thread

CAVEAT: Operations that Prevent Concurrency on the GPU

- The operations below when issued, prevent any other operation from simultaneously taking place on the GPU
  - A device memory allocation (`cudaMalloc`)
  - A device memory set (`cudaMemset`)
  - A device ↔ device memory copy (`cudaMemcpy`)
  - A CUDA command to stream 0 (including kernel launches and host ↔ device memory copies that do not specify any stream parameter)
  - A page-locked host memory allocation (`cudaMallocHost`)
  - A switch between the L1/shared memory configurations
- Most of the above hogs come with an asynchronous version suffixed by `*Async`
  - Ex: `cudaMemcpyAsync`, `cudaMemsetAsync`

Asynchronous Operations are Typically MUCH faster

- By running things asynchronously, CUDA runtime can enable many optimizations behind the scene (e.g., memory pool optimization of `cudaMallocAsync`)



Summary CUDA Stream

- CUDA streams are FIFO queues that allow applications to enable GPU task concurrency
- All GPU calls are placed into the default stream (stream ID=0) unless they are enqueued into an existing stream created from `cudaStreamCreate`
- Stream 0 is special (has special synchronization rules): **synchronous** with all streams
  - Meaning: Things done in Stream 0 cannot overlap other streams (Stream 0 is blocking)
    - Exception: Streams with non-blocking flag are exception: `cudaStreamCreate`**`WithFlags`**`(&stream, `**`cudaStreamNonBlocking`**`)`
  - Why do we have such a strange design and workaround with Stream 0…? – Legacy, when CUDA is first introduced, people did not think of GPU task concurrency

Synchronize Operations on a CUDA Stream

`cudaStreamSynchronize()` takes a stream as a parameter and halts execution on the host until all preceding commands in the given CUDA stream have completed. It can be used to synchronize the host with a specific stream, allowing other streams to continue executing on the device – *block the execution of the host until returns*

`cudaDeviceSynchronize()` halts execution on the host until all preceding commands in all CUDA streams have completed – *block the execution of the host until return*

`cudaEventRecord()` takes an event and make a record (tick) from the given CUDA stream – *halts execution within the stream*

`cudaStreamWaitEvent()` takes a CUDA stream and an event as parameters and makes all the commands added to the given stream after the call to **`cudaStreamWaitEvent()`** delay their execution until the given event has completed – *halts execution within the stream*

Example: Use of `cudaStreamWaitEvent`

- Assume `stream1` and `stream2` have been defined/initialized already
- The point of this example:
  - Use the two copy sub-engines at the same time: copy in (`stream1`), copy out (`stream2`)
  - Postpone launching of `myKernel` in `stream2` until the copy operation in `stream1` is completed

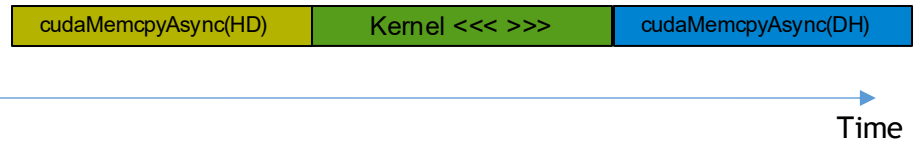
```
Repeat the following for 100 iterations:
 cudaEvent_t event;
 cudaEventCreate (&event);                                  // create event

 cudaMemcpyAsync ( d_in, in, size, H2D, stream1 );          // 1) H2D copy of new input
 cudaEventRecord (event, stream1);                          // record event

 cudaMemcpyAsync ( out, d_out, size, D2H, stream2 );        // 2) D2H copy of previous result

 cudaStreamWaitEvent ( stream2, event );                    // wait for event in stream1
 myKernel<<< 1000, 512, 0, stream2 >>> ( d_in, d_out );     // 3) GPU must wait for 1 and 2
 someCPUfunction ( blah, blahblah )                         // CPU call gets launched right away
```

[S. Rennich, NVIDIA]→

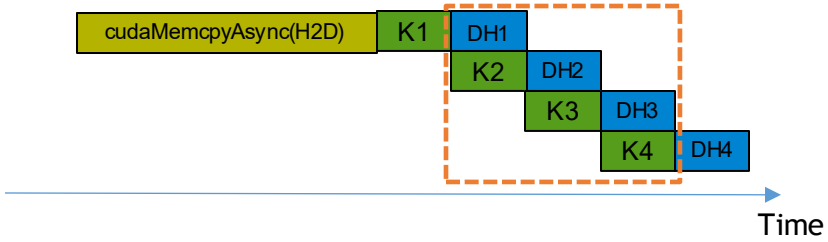
Streams Enable Pipeline-based Parallelism

A complete GPU task consists of: DH-device2host copy, K-kernel execution, and HD-host2device copy

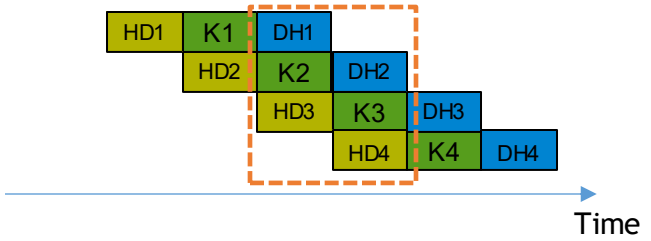
- No concurrency



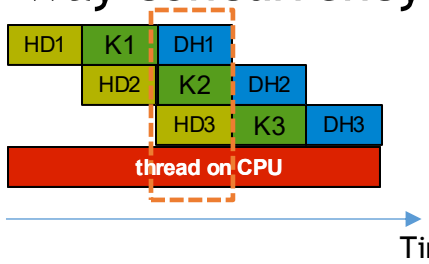
- 2-way concurrency (two streams)



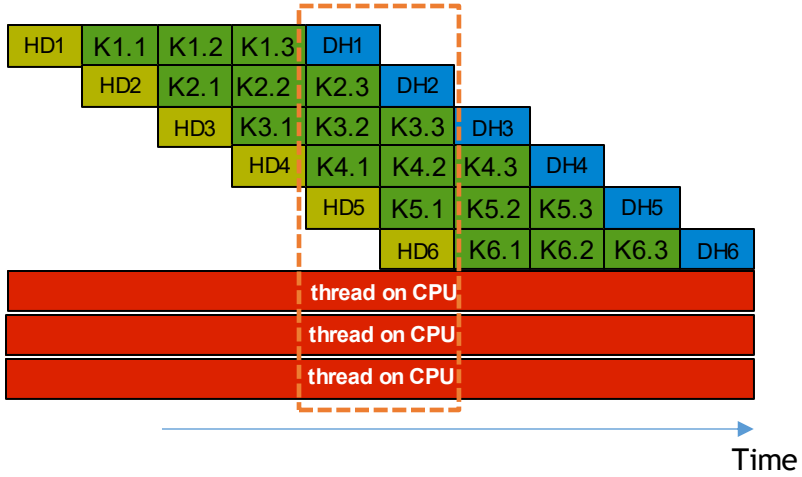
- 3-way concurrency (3 streams)



- 4-way concurrency (4 streams)



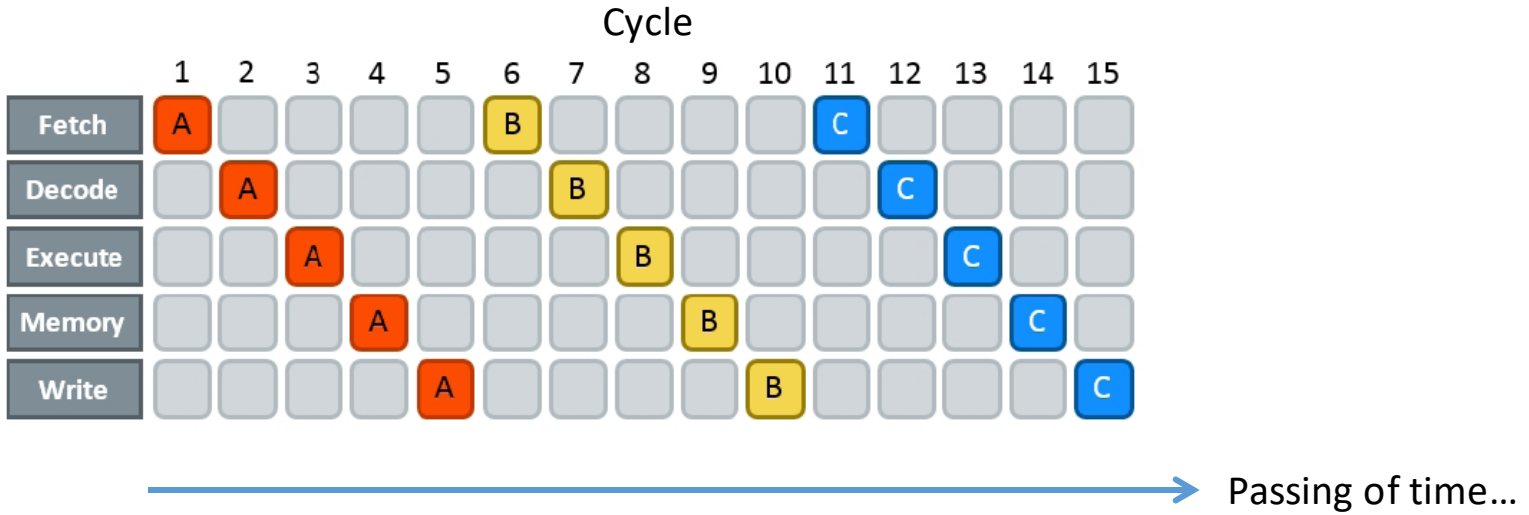
- 5+ way concurrency (5 streams)



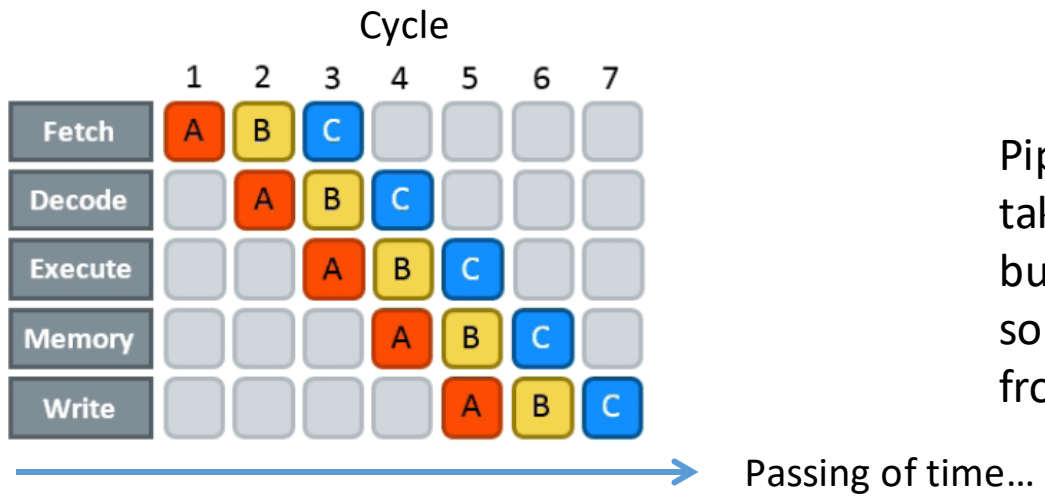
[Justin Luitjens - NVIDIA]→

Using streams is similar to Instruction-level Parallelism (ILP)

No pipelining:



With pipelining:



Pipelining: The CU takes care of business, with some early help from the compiler

[https://techdecoded.intel.io/resources/understanding-the-instruction-pipeline/#gs.665w4j]→